FIXA / MANU

Auditoria adversarial da IA e do chatbot

27 de outubro de 2026 - Relatório para correção e reteste

72 casos dirigidos. 11 pontos de correção. Um problema de prioridade funcional alta. Evidências reais de AWS separadas de falhas simuladas na interface.

Primeiro: INT-01. A confirmação de uma proposta desaparece ao recarregar a página. Não foi realizada criação operacional sem consentimento; o problema comprovado é a perda do estado e dos controles de revisão.

ID	Prioridade	Responsável	Ponto de correção
IA-01	P2	IA	Identificador com espaços causa erro 500 no encerramento
IA-02	P2	IA	Encerramento confirma sucesso para conversa inexistente ou de outra conta
IA-03	P2	IA	Conversa encerrada aceita novas mensagens
IA-04	P2	IA	Validação de imagens é mais permissiva que a da Web
IA-05	P3	IA	Resumo não fornece um título curto separado
INT-01	P1	IA + Web	Confirmação pendente desaparece ao recarregar a página
INT-02	P2	IA + Web	Imagens enviadas somem na reabertura pelo histórico
WEB-01	P2	Web	Resposta vazia é exibida como balão sem conteúdo
WEB-02	P2	Web	thread_id inválido é aceito na resposta de criação de chat
WEB-03	P2	Web	Histórico já em memória fica inacessível ao voltar à tela com API indisponível
WEB-04	P3	Web	Falha de formato do upload expõe erro técnico ao usuário

Escopo e versões

AWS: ia-api.fixa.solutions e web-api.fixa.solutions. Web local em localhost:5173, revisão 9e6434d. Perfis Gestor e RH. Código da IA consultado em leitura: main b0977fd; SHA do deploy não exposto. Sem alterações de código de produção nesta auditoria.

P1 = alta prioridade funcional; P2 = normal; P3 = acabamento/contrato. Não é escala CVSS. Os 11 pontos incluem falhas, lacunas de contrato e endurecimento de validação. Issue #12.

02 / Ciclo de vida das sessões

IA-01 | P2 | Identificador com espaços causa erro 500 no encerramento

API da IA - AWS real - Casos S07

Reprodução: Com Bearer válido, enviar DELETE /api/v1/chat/sessions/%20, sem corpo.

Observado: HTTP 500 com corpo Internal Server Error. A entrada inválida não vira um erro de validação.

Esperado: Rejeitar o identificador vazio após trim com HTTP 400 ou 422, sem executar o encerramento.

Correção: Validar e normalizar o parâmetro de caminho antes do serviço. Converter ValueError em resposta 4xx. Aplicar os mesmos limites a todas as rotas de sessão.

Aceite: Espaço, tabulação e ID vazio após normalização retornam 4xx; um ID válido continua funcionando.

app/services/sessao.py: finalizar_sessao_chat; a validação de thread_id após strip lança ValueError.

IA-02 | P2 | Encerramento confirma sucesso para conversa inexistente ou de outra conta

API da IA - AWS real - Casos S03, S05, S06

Reprodução: Gerar um UUID sem conversa e chamar DELETE /api/v1/chat/sessions/{uuid}. Repetir com o ID de uma conversa de QA da outra conta autorizada.

Observado: Ambos retornaram HTTP 200, status ENCERRADA, mensagem Sessão encerrada com sucesso e resumo null.

Esperado: Retornar 404 quando não houver conversa acessível ao usuário autenticado. Se houver política de DELETE idempotente, ela precisa ser explícita e não afirmar que houve encerramento e resumo.

Correção: Consultar a sessão por identificador e usuário antes de resumir ou responder. Manter autorização por proprietário e retornar erro uniforme para inexistente/sem acesso.

Aceite: Conversa própria encerra; conversa inexistente ou de outra conta retorna 404. Não retorna dados da outra conta nem modifica seu estado.

app/services/sessao.py e persistência de sessões. Não houve evidência de leitura ou alteração da conversa alheia: a leitura pelo dono continuou disponível e sem resumo.

IA-03 | P2 | Conversa encerrada aceita novas mensagens

API da IA - AWS real - Casos C01, C02

Reprodução: Criar conversa de QA, encerrá-la pelo DELETE de chat/sessions e enviar POST /api/v1/chat com o mesmo thread_id.

Observado: O encerramento retornou ENCERRADA com resumo. O POST posterior retornou 200, o mesmo thread_id e nova resposta: Olá! Como posso ajudar?

Esperado: Pelo fluxo atual da Web, uma conversa salva fica somente para leitura; a API deve retornar 409 ao tentar continuar. Uma política de reabertura exigiria contrato e atualização explícita do resumo.

Correção: Persistir estado explícito da conversa e verificá-lo antes de invocar o grafo. Não inferir encerramento somente da existência de texto de resumo.

Aceite: POST em sessão encerrada é recusado sem invocar ferramentas; uma conversa nova continua aceitando mensagens.

app/services/chat.py: consulta a existência e o proprietário, mas não bloqueia o estado encerrado antes do grafo.

03 / Imagens e título do resumo

IA-04 | P2 | Validação de imagens é mais permissiva que a da Web

API da IA - AWS real; lacuna de validação - Casos I01, I02, I03, U11, U12

Reprodução: Enviar uma imagem com URL file:///__fixa_qa_arquivo_inexistente__.png. Em outro envio da mesma conversa de QA, enviar quatro URLs públicas de imagem.

Observado: As duas requisições retornaram HTTP 200 e resposta genérica fora_de_escopo. A Web rejeita protocolos fora de HTTP/HTTPS e limita a três imagens; a API aceita lista de strings sem esses limites.

Esperado: Validar também no servidor o protocolo, a quantidade e o formato da URL. Definir uma política de origem e de limites de conteúdo antes de qualquer download.

Impacto: Clientes que não passam pela Web conseguem ignorar as restrições do produto. Há superfície de entrada não validada no servidor.

Correção: Alinhar o schema de imagens ao limite acordado de três anexos e validar URL. Se o serviço baixar imagens, proteger destino, redirecionamentos, timeout, tamanho e tipo do conteúdo.

Aceite: file:, javascript:, URL inválida e quatro anexos são recusados com 422; anexos válidos continuam no fluxo esperado.

app/schemas/chat.py: imagens é list[str], sem limite de itens/URI. Isto é endurecimento do contrato atual; não foi demonstrado SSRF, leitura de arquivo ou acesso à rede interna.

IA-05 | P3 | Resumo não fornece um título curto separado

API da IA - AWS real + código-fonte - Casos C01

Reprodução: Encerrar uma conversa de teste e examinar resumo; comparar com o formatador de histórico que separa a primeira linha como titulo.

Observado: A resposta veio em um único parágrafo, começando com O usuário informou que era um teste de integração [...]. Não havia título separado. O código de histórico usa a primeira linha como título.

Esperado: Devolver titulo curto e resumo em campos distintos, sem depender de Markdown, primeira linha ou interpretação pela Web.

Impacto: O cabeçalho pode receber um parágrafo. A Web tem uma proteção que usa a primeira mensagem quando o título não é utilizável, mas isso não entrega o título do resumo solicitado.

Correção: Gerar saída estruturada e validada com titulo e resumo. Devolver ambos no encerramento e nas duas consultas de histórico.

Aceite: Um resumo de várias frases produz um titulo curto independente; o mesmo título aparece após reload. Falha ao gerar resumo não muda indevidamente o estado.

app/memory/persistencia/gerar_resumo.py e app/services/sessoes/historico.py; proteção atual em src/utils/manu.ts.

Atenção à classificação

Aceitar file: não prova leitura de arquivo ou SSRF. O arquivo usado era fictício e inexistente; nenhum endereço interno ou metadado AWS foi consultado. A resposta foi genérica. A falha demonstrada é a ausência de validação equivalente à Web.

04 / Confirmação deve sobreviver ao reload

INT-01 | P1 | Confirmação pendente desaparece ao recarregar a página

API da IA + Web - Web real com contrato remoto controlado - Casos U07, B02

Reprodução: Responder ao envio com confirmacao=sim. Verificar os botões Sim e Não. Recarregar e abrir a mesma conversa, usando o DTO atual do histórico.

Observado: Antes do reload há proposta e botões. Depois há apenas um balão comum; Sim aparece zero vezes e o campo de mensagem volta a ficar habilitado. O histórico não informa a confirmação pendente nem a marcação da mensagem.

Esperado: A proposta deve continuar pendente após reload, troca de dispositivo e nova sessão de navegador. Sim/Não e a correção precisam continuar associados à proposta correta.

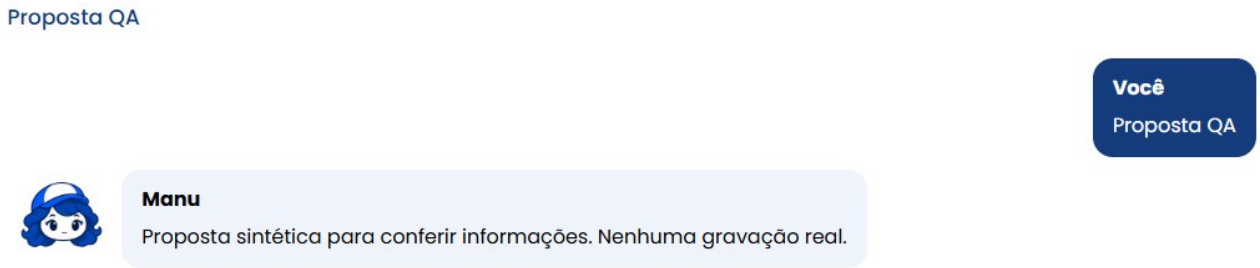
Correção: Persistir proposta, estado e identificador da ação no backend; expor estado no histórico. A Web deve restaurar os controles pelos campos remotos. A verificação de consentimento também precisa existir no servidor.

Aceite: Recarregar com proposta pendente preserva a revisão; dupla confirmação não duplica criação; confirmação de proposta antiga é rejeitada. A etapa com criação real deve ser homologada em ambiente de teste.

Contrato em src/types/api/manu.ts; src/services/manuService.ts define confirmacao=nao ao decodificar histórico. O cache só mascara a ausência enquanto a página permanece carregada.



Antes: proposta controlada com Sim/Não. Nenhuma criação real.



Depois: a mesma resposta vira balão comum e a confirmação desaparece. O campo para nova mensagem volta a ser habilitado.

05 / Contexto visual e resposta vazia

INT-02 | P2 | Imagens enviadas somem na reabertura pelo histórico

API da IA + Web - Web real com contrato remoto controlado - Casos U08, B03

Reprodução: Enviar mensagem com uma imagem, recarregar a página e abrir novamente a conversa.

Observado: A mensagem tinha uma imagem antes do reload e zero depois. O DTO remoto só contém role, mensagem e emocao; o adaptador define images=[] em cada mensagem.

Esperado: O histórico deve conservar as URLs das imagens enviadas, na mensagem original e na ordem original.

Impacto: A conversa reaberta perde contexto visual. O usuário não consegue conferir qual foto havia enviado.

Correção: Persistir e devolver imagens por mensagem e mapear esses campos na Web; validar as URLs também ao ler o histórico.

Aceite: Uma a três imagens válidas continuam visíveis após reload e em outra sessão autenticada; uma conversa sem anexos continua funcionando.

src/types/api/manu.ts e src/services/manuService.ts: decodeHistory. Upload do teste foi interceptado; nenhum arquivo real foi enviado ao Cloudinary.

WEB-01 | P2 | Resposta vazia é exibida como balão sem conteúdo

Web; contrato de saída da IA também deve validar - Resposta defeituosa controlada - Casos U01, U02, B01

Reprodução: Simular POST /api/v1/chat com resposta vazia ou somente espaços e os demais campos válidos.

Observado: O serviço aceita e armazena a resposta. A tela mostra um balão com Manu e nenhum conteúdo, sem alerta. A validação atual só exige typeof resposta=string.

Esperado: Rejeitar resposta sem conteúdo após trim, preservar o contexto e apresentar erro compreensível, sem repetir o POST automaticamente.

Impacto: A conversa parece ter sido respondida, mas o usuário não recebe informação nem indicação da falha.

Correção: Validar resposta.trim() tanto na saída da IA quanto no adaptador Web. Manter a política de não repetir ações ambíguas automaticamente.

Aceite: Strings vazias e em branco produzem erro legível, sem bolha vazia e sem reenvio automático.

src/services/manuService.ts: validação da resposta de send. Não foi observada resposta vazia da AWS nesta auditoria.



Resposta vazia simulada: a Web mostra apenas Manu e não apresenta alerta. Não ocorreu em uma resposta real da AWS nesta coleta.

06 / Validação da Web e recuperação do histórico

WEB-02 | P2 | thread_id inválido é aceito na resposta de criação de chat

Web - Resposta defeituosa controlada - Casos U03, U04

Reprodução: Simular a primeira resposta com thread_id de 101 caracteres ou somente três espaços. Tentar encerrar a conversa depois.

Observado: Ambos os IDs foram aceitos e guardados. O de 101 caracteres falha depois com Conversa inválida.; o de espaços passa da primeira validação e não tem um encerramento confirmado.

Esperado: Validar o mesmo formato e limite do identificador ao receber, enviar, ler e encerrar a conversa. Não gravar resposta incompatível no cache.

Impacto: A interface entra em um estado que não consegue operar de forma consistente nas próximas ações.

Correção: Compartilhar a validação de ID: string normalizada não vazia, limite de 100 e formato pactuado. Validar antes de remember().

Aceite: Resposta com ID vazio após trim ou acima do limite é rejeitada imediatamente; IDs válidos continuam aceitos.

src/services/manuService.ts: send verifica truthiness, enquanto close aplica limite de 100. A AWS não retornou ID inválido nos testes reais.

WEB-03 | P2 | Histórico já em memória fica inacessível ao voltar à tela com API indisponível

Web - Falha HTTP 503 controlada - Casos B04, B14

Reprodução: Conversar normalmente; navegar para Perfil; fazer a listagem da IA responder 503; voltar para Manu sem recarregar toda a aplicação.

Observado: O cache do serviço ainda tinha uma conversa, mas a barra lateral mostrava zero. Apareceu o aviso de falha, sem disponibilizar os dados já carregados. Tentar novamente recuperou a lista quando a API voltou.

Esperado: Manter as conversas em memória acessíveis junto do aviso de que não foi possível atualizá-las.

Impacto: Uma indisponibilidade temporária bloqueia o acesso ao histórico já disponível nesta sessão.

Correção: Usar cachedList também no carregamento inicial que falhou, como já é feito em refreshConversations. Identificar que os dados podem estar desatualizados.

Aceite: Na navegação Perfil > Manu com 503, a conversa em cache continua selecionável; o aviso permanece e o retry atualiza depois.

src/pages/Manu/index.tsx: catch do useEffect inicial não popula conversations; refreshConversations faz esse fallback.

Captura completa do caso B04 disponível nas evidências anexas: history-503-with-cache.png.

07 / Falhas de upload e resiliência

WEB-04 | P3 | Falha de formato do upload expõe erro técnico ao usuário

Web / integração Cloudinary - Resposta defeituosa controlada - Casos U09, U10, B10

Reprodução: Simular resposta de upload 200 contendo HTML. Alternativamente, retornar secure_url=not a url.

Observado: HTML dispara SyntaxError e a interface mostra Unexpected token [...] is not valid JSON. URL inválida gera TypeError: Invalid URL.

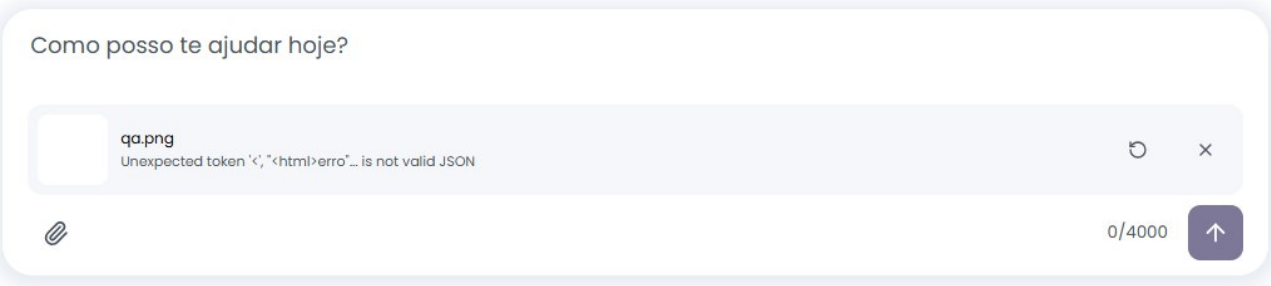
Esperado: Exibir uma mensagem curta em português sobre a falha do envio, conservando as ações de remover e tentar novamente.

Impacto: O usuário recebe detalhe técnico incompreensível em vez de orientação útil.

Correção: Proteger response.json() e new URL() no serviço e converter falhas em ApiError de aplicação. Preservar cancelamento e eventual status de erro do provedor.

Aceite: HTML, JSON malformado, secure_url ausente/inválida e erro HTTP apresentam mensagem amigável; remover/repetir continuam funcionando.

src/services/cloudinaryService.ts: response.json() e new URL(); src/pages/Manu/index.tsx: errorMessage repassa a mensagem do Error.



Upload controlado com HTML: o erro técnico de JSON fica visível ao usuário.

Observações adicionais, fora da contagem de 11

U06: uma conversa malformada derruba a listagem inteira. A API deve devolver schema válido; como resiliência adicional, avaliar manter as conversas válidas. Falha somente controlada.

A02: Bearer inválido foi recusado com 403, não 401. A autorização funcionou. Alinhar o status com a política de renovação; expiração de JWT não foi testada.

Estado explícito: hoje a Web infere encerramento de titulo/resumo. Recomenda-se status próprio no histórico, inclusive quando a geração de resumo falhar. Este último risco foi identificado em código, sem novo caso AWS reproduzido.

08 / Cobertura e comportamento preservado

Grupo	Casos	Modo
AWS / logins	41	39 ensaios IA + 2 logins; limpeza e contrato fora da conta
Adaptadores	14	Módulos de produção com transporte controlado
Interface	17	Web real, APIs e Cloudinary interceptados
Total dirigido	72	Casos únicos; repetição de roteiro não somada
Regressão existente	25 da Manu	Baseline antes dos ensaios, todos aprovados

Autorização: Sem token/esquema errado: 401. Ler, excluir ou enviar na conversa da outra conta: 404, sem dados retornados. Ressalva de falso sucesso no encerramento em IA-02.

Entradas e tamanho: 12 entradas inválidas retornaram 422. Mensagem válida de 4000 caracteres não quebrou o serviço.

Prompts e HTML: Tentativas ensaiadas de alterar o enum e revelar o token foram bloqueadas. HTML malicioso ficou como texto na Web. Isso não certifica todos os prompts.

Cliques e falhas: Duplo clique não duplicou envio/confirmação. Sem retry automático de POST. Rascunho, correção e conversas foram preservados nas falhas simuladas de envio/fechamento/exclusão.

Histórico e exclusão: Retry recuperou a listagem. Depois de excluir conversa própria, ler/enviar retornaram 404.

Mobile e arquivos: 320x667 e 844x390 sem overflow nos casos avaliados. Resposta de 10 mil caracteres coube. Arquivos vazios, acima de 5 MiB e SVG foram recusados na Web.

Limites

Sem carga/DoS, redes privadas, metadados AWS, criação de solicitações/OS/ocorrências/eventos/compras, visão ponta a ponta, todas as ferramentas, expiração temporal do token ou toda a matriz de dispositivos. As confirmações foram simuladas para não gravar registros. Nenhum vazamento entre instituições, execução de script, SSRF ou leitura de arquivo foi demonstrado. Não há garantia de ausência de outros erros.

Três conversas próprias de QA foram removidas. Nenhuma conversa anterior do usuário foi apagada. Não há tokens ou senhas nos arquivos entregues.

09 / Reprodução HTTP e ordem de reteste

Usar apenas token próprio e conversas novas de QA. Todos os DELETEs abaixo são sem corpo. Os identificadores são placeholders.

IA-01: entrada vazia após normalização

```
DELETE /api/v1/chat/sessions/%20
Authorization: Bearer <TOKEN_QA>

Observado: 500. Esperado: 400/422.
```

IA-02: conversa inexistente

```
DELETE /api/v1/chat/sessions/<UUID_NOVO_SEM_CONVERSA>
Authorization: Bearer <TOKEN_QA>

Observado: 200 ENCERRADA / resumo null. Esperado: 404.
```

IA-03: ciclo de vida

```
POST /api/v1/chat
Authorization: Bearer <TOKEN_QA>
Content-Type: application/json

{"mensagem":"Olá, teste sem criar registros.",
 "thread_id":null,"imagens":[]}

DELETE /api/v1/chat/sessions/<THREAD_ID_RETORNADO>
Authorization: Bearer <TOKEN_QA>

POST /api/v1/chat com o mesmo thread_id.
Observado: 200. Esperado no fluxo atual: 409.
```

Reteste sugerido

1. Restaurar confirmação/anexos no contrato. 2. Corrigir validação e ciclo de vida da API. 3. Corrigir robustez da Web. 4. Repetir os critérios de cada item e os testes positivos. 5. Homologar criação real e idempotência em ambiente próprio de teste.

10 / Contrato, evidências e entrega

Campos a combinar entre IA e Web

Devolver estado explícito da sessão, titulo separado do resumo, URLs por mensagem e confirmação pendente associada a uma proposta estável. Persistir esses dados no backend; o cache da página não deve ser a única fonte.

```
Exemplo para discussão, ainda não implementado:
{
 "session_id": "id-estavel",
 "status": "ATIVA",
 "titulo": "Manutenção do ar-condicionado",
 "resumo": "Texto separado do título.",
 "confirmacao_pendente": true,
 "proposta_id": "acao-a-revisar",
 "mensagens": [
 {"role":"human", "mensagem":"Veja a foto",
 "imagens":["https://res.cloudinary.com/.../foto.jpg"]},
 {"role":"assistant", "mensagem":"Confira a proposta",
 "emocao":"CONFIRMACAO", "confirmacao":"sim",
 "proposta_id":"acao-a-revisar"}
 ]
}
```

A confirmação deve ser validada no servidor, com proteção contra repetição e proposta desatualizada. A perda do botão na Web não prova que a IA execute ações sem consentimento.

Evidências entregues

Relatório Markdown: docs/AUDITORIA_MANU_2026-10-27.md. JSONs e seis capturas: docs/evidencias/manu-auditoria-2026-10-27/. Os resultados brutos têm expectativas de teste; a classificação deste relatório prevalece sobre contagens automáticas de pass=false.

Referências principais

API - chat - abrir código na revisão consultada

API - schemas - abrir código na revisão consultada

API - encerramento - abrir código na revisão consultada

API - resumo - abrir código na revisão consultada

API - histórico - abrir código na revisão consultada

Web - adaptador - abrir código na revisão consultada

Web - tela - abrir código na revisão consultada

Web - upload - abrir código na revisão consultada

Validação e registro

Format, lint e build aprovados; 117 testes existentes aprovados. Resultados registrados em resumo.json. Nenhum código de produção da IA ou Web foi modificado. Sem PR, merge ou deploy nesta atividade.

Início observado: 16h41m03s. Documento gerado às 17h14m18s (São Paulo), em 27/10/2026. Estimativa inicial: 90 min. Tempo decorrido até a geração: 33.3 min. O registro final de entrega consta em resumo.json.